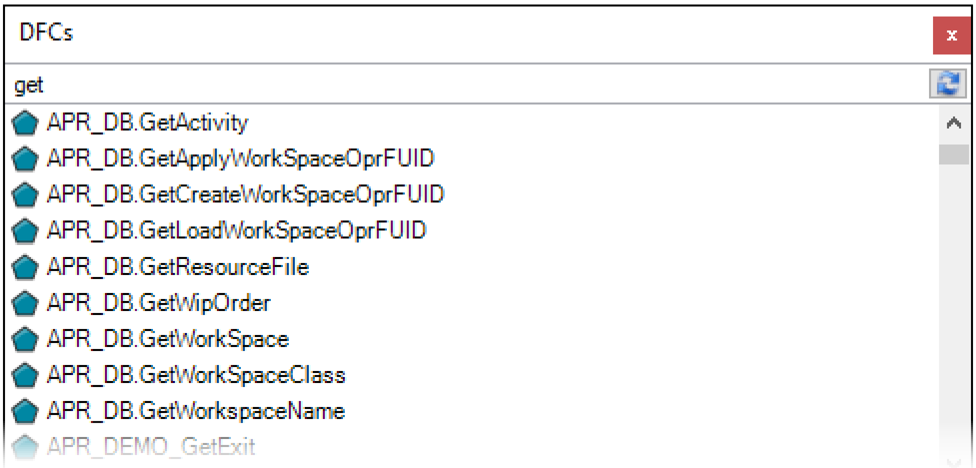
Sub-DFC Function

The Sub-DFC function executes a DFC. The DFC to execute can be selected by name (and optionally by revision), or determined by the FUID.

To Add the Sub-DFC Function

1. Drag the Sub-DFC function from the toolbox to the function navigation diagram, or double-click the icon in the toolbox.
2. The system displays a list of sub-DFCs that the user is able to choose from. When the user enters a part of a DFC code, the system suggests a list of matching entries from the database.



> The pop-up window displays names of DFCs and additionally project codes (only for DFCs that exist in projects).

Optionally, choose the revision of the DFC. By default, the revision will be determined dynamically – the system uses the default revision.

3. After selecting a sub-DFC, the system adds inputs and outputs to the function based on the external inputs and outputs of the chosen DFC.

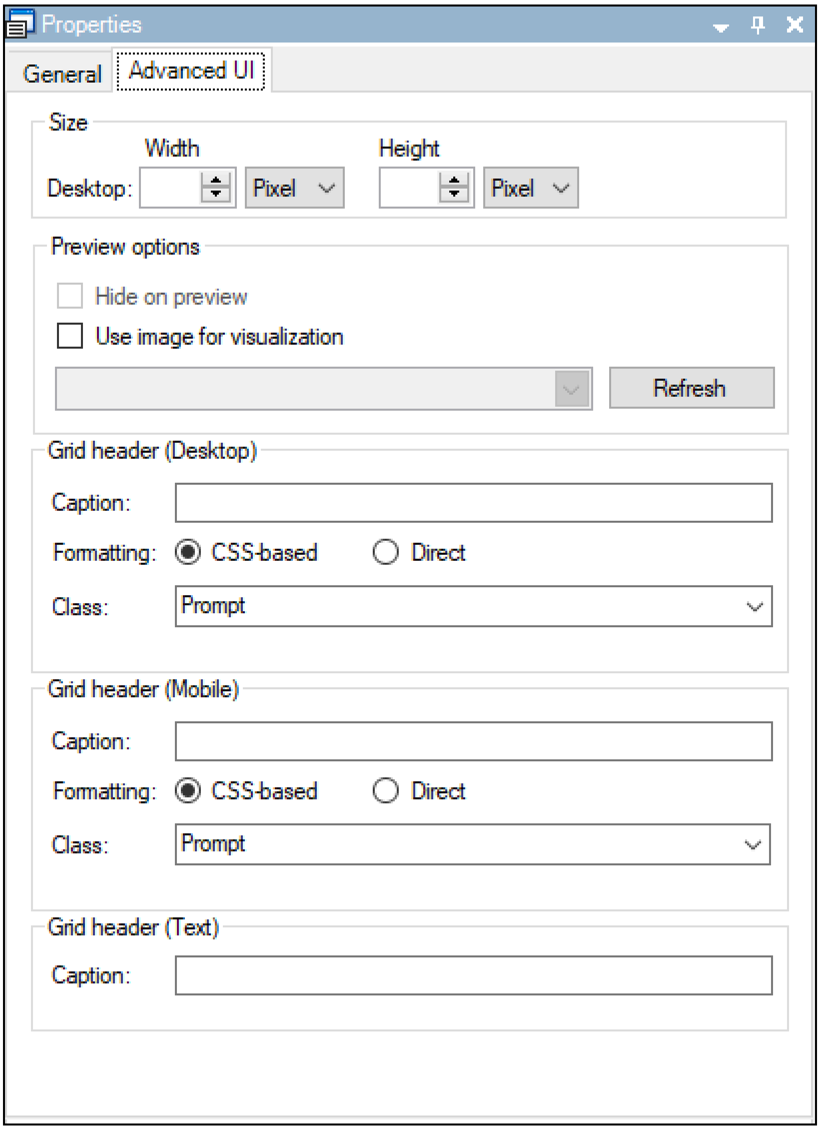
General Tab

Field	Description
Selected by code	If selected, the DFC will be determined by the DFC code and optionally by the revision.
Code	The code of the linked DFC that will be executed in runtime.
Revision	If chosen, a specific revision of the DFC will be executed. Otherwise, the default revision will be chosen. In the case of project DFCs, the system always uses the default revision.
Open	Opens the linked DFC in a new tab.
Link	Links a DFC. DFCs available in Entity Manager cannot be linked as sub-DFCs for DFCs available in PB projects.

	In PB projects, you can link predefined DFCs or those existing only in the default and non-default revisions of a given project.
Copy and Link	Displays a list of sub-DFCs that the user is able to link, duplicates the selected DFC, and links it.
Refresh Inputs and Outputs	Refreshes the list of external inputs and outputs of the linked DFC, and updates them in the function workspace.
Determined by FUIDs from Function Inputs	If selected, the DFC will be determined dynamically by the FUID numbers that are passed from the function inputs, which are automatically created when you select this option. The inputs are: - *StaticDFCRevisionFUID* – the FUID of the default or non-default DFC revision taken from the DFC_REVISION table. - *DynamicDFCFUID* – the FUID taken from the DFC table. It automatically determines the default DFC revision or non-default revision that is assigned for specific employees if such a project-employee configuration exists and is active. The data types of both inputs must be *Char* or *List of Char (Iterate)*. The *SubOperationFUID* function input was used previously and might still be available when deploying entities from previous DELMIA Apriso versions. The input works the same way as *StaticDFCRevisionFUID*.
DFC External Inputs and Outputs	The user is able to choose which inputs and outputs will be bound to the function.
Execution Mode	There are three types of Execution Modes to choose from: - **Synchronous** – the sub-DFC is executed immediately, and further functions will not be executed until it is finished - **Asynchronous, immediate** – executes the sub-DFC in Job Executor - **Asynchronous, scheduled** – executes the sub-DFC in Job Scheduler at a defined time, and a new function input is automatically created to specify the date of execution
Pool name	The name of the Pool configured in Job Executor. Different Pools can have different priorities.
Use Pool Name from Input	If selected, the name of the Pool from the input will be used.
Synchronization Queue	The user-defined name of the synchronization queue, which guarantees that the background jobs will be executed in the same order as they were created.
Use Synchronization Queue Name from Input	If selected, the name of the Synchronization Queue from the input will be used.
Retries	The number of execution retries for the DFC after it fails.
Sleep Time	The time the execution of the DFC is postponed between retries (in seconds).
Remote - other Apriso instance (deprecated)	This functionality is deprecated and should not be used anymore. By choosing this option you can invoke DFC on another Apriso Server. Invoked DFC must be configured as Web Service on remote server. A new input (of the

	Char type) named *RemoteServerName* is created. The name of the remote server should be passed as the value of this Input. In the synchronous mode, the Sub-DFC function invokes dedicated Web Service on a remote machine under the following URL: `http://[RemoteServerName]/Apriso/WebServices/FlexNetOperationsService.svc` In the asynchronous mode, the local Job Scheduler receives a task to call the above Web Service in the background. By default, the FlexNetOperationsService in `<drive>\Program Files\Dassault Systemes\DELMIA Apriso 2025\WebSite\WebServices\Web.config` is put in comments. To invoke an DFC execution on a remote server, the section should be uncommented. To enable outgoing requests from the source DELMIA Apriso server, `RemoteServersForOperations` parameter must be configured in `CentralConfiguration.xml`.
Timeout	The execution timeout for the sub-DFC. The default value is 30 minutes. This value can only be changed for Asynchronous Execution Modes.

Advanced UI Tab



> Mobile and Text platforms are no longer supported.

Field	Description
Size	Enables the defining of the display size of the control. The Sub-DFC function will be shown in the Layout Editor with the size as defined in this section.
Hide on preview	If selected, the Sub-DFC function will not be displayed on the Preview tool pane. If a sub-DFC does not have any User Interface, select **Hide on preview** check box.
Use image for visualization	If selected, there is a possibility to display image visualizing Sub-DFC function in Layout Editor or HTML Layout Editor. This option is not available when the Hide on preview check box is selected.
Grid header (Desktop)/Grid header (Mobile)	**Caption** – the name of the header that will be displayed in runtime. Formatting ▸ **CSS-based** – the user may specify a single CSS class (by

	selecting an item from the drop-down list or by typing its name), or type multiple classes (space separated) to be used for the following Prompt element ▸ **Direct**– allows the user to define the fonts, colors, and alignment of the header **Class** drop-down list – the list of predefined classes.
Grid header (Text)	**Caption** – the name of the header that will be displayed in runtime.

Sub-DFC Function Inputs

Input	Description
ExecutionTime	The time when the DFC will be executed. The passed value will be converted to and stored as UTC. The input is available when the Execution Mode is set to asynchronous and scheduled.
RemoteServerName	The name of the remote server on which the DFC will be executed. This is available when the **Remote - other Apriso instance (deprecated)** check box is selected. The *RemoteServerName* input should not be used anymore because the functionality of invoking DFC on another Apriso server is deprecated.

Passing Dynamic Parameters

In some advanced configurations the need of passing parameters dynamically between DFCs arises. It usually happens when:

- There are many sub-DFCs launched one after another.
- Part of the interface (External Inputs/Outputs) between DFCs is static and part is dynamic.
- The interface may change throughout life cycle of the solution.
- Sub-DFC is called dynamically (by FUID) and exact inputs and outputs are not known by the caller and may change depending on a sub-DFC.

Example 1

To understand this functionality better analyze the following example.

Imagine such configuration:

- DFC **Opr01** calls **Opr02** (the main DFC)
- DFC **Opr02** calls **Opr03** with external inputs (AAA) and outputs (AAA)
- DFC **Opr03** with external inputs (AAA) and outputs (AAA) (the last DFC)

1. If you want to pass a parameter (EEE) from **Opr01** and then read its value in last DFC **Opr03** (and reverse) then you would have to change interface in all DFCs.
   Configuration would look as follows:
   - DFC **Opr01** calls **Opr02**
   - DFC **Opr02** calls **Opr03** with external inputs (AAA, EEE) and external outputs (AAA, EEE)
   - DFC **Opr03** with external inputs (AAA, EEE) and external outputs (AAA, EEE)

   > Note that parameter (**EEE**) is only needed to be used in **Opr01** and **Opr03**, but interface was changed everywhere.

2. If you want to pass another parameter (FFF) from **Opr01** and then read its value in last DFC **Opr03** then you would have to change interface in all DFCs again.
   Configuration would look as follows:
   - DFC **Opr01** calls **Opr02**
   - DFC **Opr02** calls **Opr03** with external inputs (AAA, EEE,FFF) and external outputs (AAA, EEE, FFF)
   - DFC **Opr03** with external inputs (AAA, EEE,FFF) and external outputs (AAA, EEE, FFF)

   > Note that parameters EEE and FFF are only needed to be used in **Opr01** and **Opr03**, but your interface grows unnecessarily in all DFCs.

This is a simple example, but imagine how such a design would look like if you had more than 3 DFCs, more than 3 parameters and sub-DFCs called dynamically by FUIDs.

Dynamic Parameters support and simplify such scenarios as above.

First define static and dynamic part of your interface.

- Static interface are parameters that are used in all DFCs.
- Dynamic part of interface are parameters used only in some DFCs.

Two methods can be used to pass Dynamic Parameters:

Method 1 (Recommended): Using Complex data type.

Method 2 (Deprecated): Using serialized data.